

C# and ADO.NET – Module 3 Answers

Exercise 1: Set Up the Development Environment

Program.cs

```
// Exercise 1: Hello World - validate environment
using System; class Program { static void
Main(string[] args) {
    Console.WriteLine("Hello, World!"); Console.WriteLine($"Runtime:
    {System.Runtime.InteropServices.RuntimeInformation.FrameworkDescription}");
}
}
```

[Console Output – Exercise 1]

```
Hello, World!
Runtime: .NET 8.0.x
```

Exercise 2: Value vs Reference Types

Program.cs

```
using System; class MyClass { public int Value; }
class Program { static void ModifyValue(int x) { x =
100; } static void ModifyRef(MyClass obj) { obj.Value
= 100; } static void Main() { // Value type int num =
10; Console.WriteLine($"Before ModifyValue: {num}");
ModifyValue(num); Console.WriteLine($"After ModifyValue:
{num}"); // still 10

// Reference type
MyClass obj = new MyClass { Value = 10 };
Console.WriteLine($"Before ModifyRef: {obj.Value}");
ModifyRef(obj); Console.WriteLine($"After ModifyRef:
{obj.Value}"); // now 100
}
}
```

[Console Output – Exercise 2]

```
Before ModifyValue: 10
After ModifyValue: 10
Before ModifyRef: 10
After ModifyRef: 100
```

Exercise 3: Primary Constructors in C# 12

Program.cs

```
using System;

// C# 12 primary constructor syntax class Person(string
firstName, string lastName, int age)
{
    public string FirstName { get; } = firstName; public string
    LastName { get; } = lastName; public int Age { get; } = age;
    public void DisplayInfo() => Console.WriteLine($"Name:
    {FirstName} {LastName}, Age: {Age}"); } class Program { static
    void Main() { var p = new Person("Alice", "Smith", 30);
    p.DisplayInfo(); Console.WriteLine($"Full Name:
    {p.FirstName} {p.LastName}");
    }
}
```

[Console Output – Exercise 3]

```
Name: Alice Smith, Age: 30
Full Name: Alice Smith
```

Exercise 4: Type Inference with var and new()

Program.cs

```
using System; class Point { public int X; public
int Y; } class Program { static void Main() { var
num = 42; // int var text = "Hello"; // string var
price = 9.99; // double Point pt = new() { X=1, Y=2
}; // new() target-typed

Console.WriteLine($"num: {num} ({num.GetType().Name})");
Console.WriteLine($"text: {text} ({text.GetType().Name})");
Console.WriteLine($"price: {price} ({price.GetType().Name})");
Console.WriteLine($"pt: X={pt.X} Y={pt.Y} ({pt.GetType().Name})");
}
}
```

[Console Output – Exercise 4]

```
num: 42 (Int32)
text: Hello (String)
price: 9.99 (Double)
pt: X=1 Y=2 (Point)
```

Exercise 5: Conditional Logic for Grade Calculation

Program.cs

```
using System; class Program { static void Main()
{ Console.Write("Enter score (0-100): "); int
score = int.Parse(Console.ReadLine() ?? "0");
// if/else chain string grade; if (score >=
90) grade = "A"; else if (score >= 80) grade
= "B"; else if (score >= 70) grade = "C";
else if (score >= 60) grade = "D"; else
grade = "F"; Console.WriteLine($"if/else
grade: {grade}");

// switch with pattern
matching string swGrade =
score switch {
>= 90 => "A",
>= 80 => "B",
>= 70 => "C",
>= 60 => "D",
_ => "F"
}; Console.WriteLine($"switch grade:
{swGrade}");
}
}
```

[Console Output – Exercise 5 (score = 85)]

```
Enter score (0-100): 85
if/else grade: B switch
grade: B
```

Exercise 6: Loop Through an Array with Different Loop Types

Program.cs

```
using System; class Program { static void
Main() { int[] nums = { 1, 2, 3, 4, 5, 6, 7,
8, 9, 10 };
// for loop - skip even numbers
Console.Write("for (odd only): ");
for (int i = 0; i < nums.Length;
i++) { if (nums[i] % 2 == 0)
continue; Console.Write(nums[i] + "
");
}
Console.WriteLine();
// foreach loop - stop at 6
Console.Write("foreach (stop@6):
"); foreach (int n in nums) { if (n
== 6) break; Console.Write(n + "
");
}
Console.WriteLine();
// while loop
Console.Write("while: "); int
idx = 0; while (idx <
nums.Length)
Console.Write(nums[idx++] + "
");
Console.WriteLine();
// do-while loop
Console.Write("do-while: ");
int j = 0; do {
Console.Write(nums[j++] + "
");
} while (j < nums.Length);
Console.WriteLine();
}
}
```

[Console Output – Exercise 6]

```
for (odd only): 1 3 5 7 9
foreach (stop@6): 1 2 3 4 5
while: 1 2 3 4 5 6 7 8 9 10 do-
while: 1 2 3 4 5 6 7 8 9 10
```

Exercise 7: Method Overloading

Program.cs

```

using System; class Program { static int CalculateTotal(int a, int b)
=> a + b; static double CalculateTotal(double a, double b) => a + b;
static double CalculateTotal(double a, double b, double c) => a + b +
c; static string CalculateTotal(string a, string b) => a + b; static
void Main() {
Console.WriteLine($"int + int: {CalculateTotal(3, 4)}");
Console.WriteLine($"double + double: {CalculateTotal(2.5, 3.5)}");
Console.WriteLine($"3 doubles: {CalculateTotal(1.1, 2.2, 3.3)}");
Console.WriteLine($"string concat: {CalculateTotal("Hello, ", "World!")}");
}
}

```

[Console Output – Exercise 7]

```

int + int: 7 double + double:
6 3 doubles:
6.6000000000000005 string
concat: Hello, World!

```

Exercise 8: ref, out, and in Parameters

Program.cs

```

using System; class Program { // ref: caller must
initialise; method can read AND write static void
DoubleIt(ref int x) => x *= 2;
// out: caller need not initialise; method MUST assign before
return static void GetMinMax(int[] arr, out int min, out int max) {
min = arr[0]; max = arr[0]; foreach (int n in arr) { if (n < min)
min=n; if (n > max) max=n; } }
// in: read-only reference - method cannot modify
static void PrintArea(in double width, in double
height) => Console.WriteLine($"Area = {width *
height}"); static void Main() { // ref int val = 5;
Console.WriteLine($"ref before: {val}");
DoubleIt(ref val);
Console.WriteLine($"ref after: {val}");
// out int[] data = { 4, 7, 1, 9, 3 };
GetMinMax(data, out int mn, out int mx);
Console.WriteLine($"out min={mn}
max={mx}");
// in double w = 4.0, h
= 6.5; PrintArea(in w,
in h);
}
}

```

```
}
```

[Console Output – Exercise 8]

```
ref before: 5  
ref after: 10  
out min=1 max=9  
Area = 26
```

Exercise 9: Local Functions

Program.cs

```
using System; class Program { static long CalculateFactorial(int n) { // Local function - only visible inside CalculateFactorial long Factorial(int x) => x <= 1 ? 1 : x * Factorial(x - 1); if (n < 0) throw new ArgumentException("n must be >= 0"); return Factorial(n); } static void Main() { for (int i = 0; i <= 10; i++) Console.WriteLine($"{i}! = {CalculateFactorial(i)}"); } }
```

[Console Output – Exercise 9]

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Exercise 10: OOP Basics with Constructors

Program.cs

```
using System; class Car { public string Make { get; set; } public string Model { get; set; } public int Year { get; set; } // Default constructor public Car() { Make = "Unknown"; Model = "Unknown"; Year = 2000; } // Parameterised constructor public Car(string make, string model, int year) { Make = make; Model = model; Year = year; } public override string ToString() => $"{Year} {Make} {Model}"; } class Program { static void Main() { Car c1 = new Car(); Car c2 = new Car("Toyota", "Camry", 2024); Console.WriteLine($"Car 1: {c1}"); Console.WriteLine($"Car 2: {c2}"); } }
```

[Console Output – Exercise 10]

```
Car 1: 2000 Unknown Unknown
Car 2: 2024 Toyota Camry
```

Exercise 11: Access Modifiers

Program.cs

```
using System; class Animal { public string Name = "Animal"; //  
accessible everywhere private int _secret = 42; // only inside Animal  
protected string Species = "Unknown"; // Animal + derived classes  
public void ShowPrivate() => Console.WriteLine($"Private:  
{_secret}"); } class Dog : Animal { public void Show() {  
Console.WriteLine($"Name (public): {Name}");  
Console.WriteLine($"Species (protected): {Species}");  
// Console.WriteLine(_secret); // ERROR - private  
} } class Program  
{ static void  
Main() {  
Animal a = new Animal(); Console.WriteLine($"From  
outside: {a.Name}"); // OK  
a.ShowPrivate(); // OK via public method  
// a._secret - ERROR  
  
Dog d = new Dog();  
d.Show();  
}  
}
```

[Console Output – Exercise 11]

```
From outside: Animal  
Private: 42  
Name (public): Animal  
Species (protected): Unknown
```

Exercise 12: Auto-Properties and Backing Fields

Program.cs


```
using System; class Product { // Auto-  
    implemented property public string Name {  
        get; set; } = "Unknown";  
    // Backing field with validation private decimal _price;  
    public decimal Price { get => _price; set { if (value <  
        0) throw new ArgumentException("Price cannot be  
        negative.");  
        _price = value;  
    }  
}  
} } class Program  
{ static void  
Main() {  
    Product p = new Product { Name = "Laptop", Price = 999.99m  
}; Console.WriteLine($"Name: {p.Name}, Price: {p.Price:C}");  
try { p.Price = -50; } catch (ArgumentException ex) {  
    Console.WriteLine($"Error: {ex.Message}"); }  
}  
}
```

[Console Output – Exercise 12]

```
Name: Laptop, Price: $999.99  
Error: Price cannot be negative.
```

Exercise 13: Records with init Properties

Program.cs

```
using System;

// Immutable record - all properties are init-only record
Employee { public int Id { get; init; } public string Name {
get; init; } = ""; public string Department { get; init; } = "";
} class Program { static void Main() { var emp1 = new Employee {
Id=1, Name="Alice", Department="IT" };
Console.WriteLine($"Original: {emp1}");

// with-expression creates a modified COPY - original unchanged
var emp2 = emp1 with { Department = "HR" };
Console.WriteLine($"Modified copy: {emp2}");
Console.WriteLine($"Original still: {emp1}");

// emp1.Name = "Bob"; // COMPILE ERROR - init only
}
}
```

[Console Output – Exercise 13]

```
Original: Employee { Id = 1, Name = Alice, Department = IT }
Modified copy: Employee { Id = 1, Name = Alice, Department = HR }
Original still: Employee { Id = 1, Name = Alice, Department = IT }
```

Exercise 14: Inheritance and Method Overriding

Program.cs

```
using System; class Shape { public virtual void Draw() =>
Console.WriteLine("Drawing a generic shape"); } class Circle :
Shape { public override void Draw() =>
Console.WriteLine("Drawing a Circle"); } class Rectangle : Shape
{ public override void Draw() => Console.WriteLine("Drawing a
Rectangle"); } class Program { static void Main() { Shape[]
shapes = { new Shape(), new Circle(), new Rectangle() }; foreach
(var s in shapes) s.Draw(); }
}
```

[Console Output – Exercise 14]

```
Drawing a generic shape
Drawing a Circle
Drawing a Rectangle
```

Exercise 15: Abstract Classes and Interfaces

Program.cs

```
using System; abstract class Vehicle { public abstract void
Drive(); public void FuelUp() => Console.WriteLine("Fuelling
up..."); } interface IDrivable { void Start(); } class Car :
Vehicle, IDrivable { public override void Drive() =>
Console.WriteLine("Car is driving"); public void Start() =>
Console.WriteLine("Car engine started"); } class Program { static
void Main() { Car car = new Car(); car.Start(); // from interface
car.Drive(); // overridden abstract method car.FuelUp(); //
inherited concrete method

// Polymorphism
Vehicle v = new
Car();
v.Drive();
}
}
```

[Console Output – Exercise 15]

```
Car engine started
Car is driving Fuelling up...
Car is driving
```

Exercise 16: Handle Null References Safely

Program.cs

```
using System; class Person { public
string? Name { get; set; } public string?
Address { get; set; } } class Program {
static void Main() { Person? p1 = new
Person { Name = "Alice" };
Person? p2 = null;

// ?. null-conditional operator
Console.WriteLine($"p1 name: {p1?.Name ?? "No name"}");
Console.WriteLine($"p2 name: {p2?.Name ?? "No name"}");
// Chaining with ??
```

```
string address = p1?.Address ?? "Address not provided";  
Console.WriteLine($"p1 address: {address}");  
  
// Null check if (p1?.Name is not null)  
Console.WriteLine($"Name length:  
{p1.Name.Length}");  
}  
}
```

[Console Output – Exercise 16]

```
p1 name: Alice p2 name: No name p1 address:  
Address not provided  
Name length: 5
```

Exercise 17: Null-Conditional Chaining in a Contact App

Program.cs

```
using System; using System.Collections.Generic; class Contact {
public string? Name { get; set; } public string? PhoneNumber {
get; set; } } class Program { static void Main() {
List<Contact?> contacts = new() { new Contact { Name = "Alice",
PhoneNumber = "555-1234" }, new Contact { Name = null }, null };
foreach (var c in contacts) { // Safe chain: c?.Name won't throw
if c is null string display = c?.Name ?? "(no name)"; string
phone = c?.PhoneNumber ?? "(no phone)";
Console.WriteLine($"Contact: {display} | Phone: {phone}");
}
}
}
```

[Console Output – Exercise 17]

```
Contact: Alice | Phone: 555-1234
Contact: (no name) | Phone: (no phone)
Contact: (no name) | Phone: (no phone)
```

Exercise 18: The required Modifier (C# 12)

Program.cs

```
using System; class Student { public required string Name { get; init; }
public required int Grade { get; init; } public string? Notes { get;
init; } // optional } class Program { static void Main() { // All
required properties must be set in the object initialiser var s = new
Student { Name = "Bob", Grade = 10, Notes = "Good" };
Console.WriteLine($"Name: {s.Name}, Grade: {s.Grade}, Notes:
{s.Notes}");
// Leaving out Name or Grade causes a COMPILE error:
// var s2 = new Student { Grade = 10 }; //
CS9035: Required member 'Name' must be set
}
}
```

[Console Output + Compiler Behaviour – Exercise 18]

```
Name: Bob, Grade: 10, Notes: Good
Compiler error if required property is missing: CS9035:
Required member 'Student.Name' must be set in the object
initializer or attribute constructor.
```

Exercise 19: Lists and Dictionaries

Program.cs

```
using System; using
System.Collections.Generic;
class Program { static void
Main() {
// List<string> List<string> fruits = new() { "Apple",
"Banana", "Cherry" }; fruits.Add("Mango");
fruits.Remove("Banana"); Console.WriteLine("Fruits:");
foreach (var f in fruits) Console.WriteLine($" {f}");

// Dictionary<int, string>
Dictionary<int, string> employees =
new()
{
{ 101, "Alice" },
{ 102, "Bob" }, { 103, "Carol" } };
employees[104] = "Dave";
employees.Remove(102);
Console.WriteLine("Employees:"); foreach (var
kv in employees) Console.WriteLine($" ID
{kv.Key}: {kv.Value}");
}
}
```

[Console Output – Exercise 19]

```
Fruits:
Apple
Cherry
Mango Employees:
ID 101: Alice
ID 103: Carol
ID 104: Dave
```

Exercise 20: LINQ for Filtering and Projection

Program.cs

```
using System; using System.Collections.Generic; using
System.Linq; class Order { public int OrderId { get;
set; } public string CustomerName { get; set; } = "";
public double TotalAmount { get; set; } } class
Program { static void Main() { List<Order> orders =
new()
{
```

```
new Order { OrderId=1, CustomerName="Alice", TotalAmount=250.0
}, new Order { OrderId=2, CustomerName="Bob", TotalAmount=80.0
}, new Order { OrderId=3, CustomerName="Carol",
TotalAmount=450.0 }, new Order { OrderId=4, CustomerName="Dave",
TotalAmount=120.0 }, };

// LINQ: filter > 100, project to anonymous type, order by
amount var result = orders .Where(o => o.TotalAmount > 100)
.OrderByDescending(o => o.TotalAmount)
.Select(o => new { o.OrderId, o.CustomerName, o.TotalAmount });

Console.WriteLine("Orders over $100 (highest first):"); foreach (var r
in result) Console.WriteLine($" [{r.OrderId}] {r.CustomerName}:
${r.TotalAmount}");
}
}
```

[Console Output – Exercise 20]

```
Orders over $100 (highest first):
[3] Carol: $450
[1] Alice: $250
[4] Dave: $120
```

Exercise 21: Pattern Matching with is and switch

Program.cs

```
using System; class Program { static void Describe(object obj) { // is
pattern if (obj is int i) Console.WriteLine($"int: {i}"); if (obj is
string s) Console.WriteLine($"string: {s} (len {s.Length})");

// switch expression pattern matching
string kind = obj switch { int n when
n < 0 => "negative integer", int n =>
"positive integer", string { Length: 0
} => "empty string", string => "non-
empty string", null => "null", _ =>
"something else"
}; Console.WriteLine($" =>
{kind}"); } static void Main() {
Describe(42);
Describe(-7);
Describe("Hello");
Describe("");
Describe(null!);
}
}
```

[Console Output – Exercise 21]

```
int: 42 => positive integer int: -7 =>
negative integer string: Hello (len 5) =>
non-empty string string: (len 0) => empty
string
=> null
```

Exercise 22: Tuples

Program.cs

```
using System; class Program { // Method returning a named
tuple static (int Count, string Summary) AnalyseData(int[]
data) { int total = 0; foreach (int n in data) total += n;
return (data.Length, $"Sum={total}, Avg={total/data.Length}");
}
```



```
static void Main() { int[] nums = {
10, 20, 30, 40, 50 };
// Deconstruct the tuple var (count,
summary) = AnalyseData(nums);
Console.WriteLine($"Count: {count}");
Console.WriteLine($"Summary:
{summary}");
// Also accessible by name var result =
AnalyseData(nums); Console.WriteLine($"Count via
name: {result.Count}");
}
}
```

[Console Output – Exercise 22]

```
Count: 5
Summary: Sum=150, Avg=30
Count via name: 5
```

Exercise 23: Async File Upload with Exception Handling

Program.cs

```
using System; using System.Threading.Tasks; class Program { static async
Task<string> SimulateUploadAsync(string fileName, bool fail = false) {
Console.WriteLine($"Uploading {fileName}..."); await Task.Delay(3000); //
simulate 3-second upload if (fail) throw new Exception("Network error during
upload"); return $"{fileName} uploaded successfully."; } static async Task
Main() { // Success case try { string msg = await
SimulateUploadAsync("report.pdf"); Console.WriteLine(msg); } catch (Exception
ex) { Console.WriteLine($"Error: {ex.Message}");
}
// Failure case try { string msg = await
SimulateUploadAsync("data.csv", fail: true);
Console.WriteLine(msg); } catch (Exception ex) {
Console.WriteLine($"Error: {ex.Message}");
}
}
}
```

[Console Output – Exercise 23]

```
Uploading report.pdf... (3 second wait) report.pdf uploaded
successfully. Uploading data.csv...
(3 second wait)
Error: Network error during upload
```

Exercise 24: Serialize and Deserialize JSON

Program.cs

```
using System; using System.IO; using
System.Text.Json; class User { public
string Name { get; set; } = ""; public
int Age { get; set; } }
```

```

public string Email { get; set; } = ""; } class Program { static void
Main() { var user = new User { Name="Alice", Age=30,
Email="alice@example.com" };

// Serialize to JSON string var options = new
JsonSerializerOptions { WriteIndented = true }; string json =
JsonSerializer.Serialize(user, options);
Console.WriteLine("Serialized JSON:");
Console.WriteLine(json);

// Save to file
File.WriteAllText("user.json", json);
Console.WriteLine("Saved to user.json");

// Deserialize back string fileJson = File.ReadAllText("user.json"); User? loaded
= JsonSerializer.Deserialize<User>(fileJson); Console.WriteLine($"Deserialized:
{loaded?.Name}, {loaded?.Age}, {loaded?.Email}");
}
}

```

[Console Output – Exercise 24]

```

Serialized JSON:
{
  "Name": "Alice",
  "Age": 30, "Email":
  "alice@example.com"
}
Saved to user.json
Deserialized: Alice, 30, alice@example.com

```

Exercise 25: FileStream and MemoryStream

Program.cs

```

using System;
using System.IO;
using
System.Text;
class Program {
static void
Main() {
// Write a file to read back
File.WriteAllText("sample.txt", "Hello from FileStream!");

// FileStream - read text from file using (FileStream fs = new
FileStream("sample.txt", FileMode.Open)) { byte[] buffer = new
byte[fs.Length]; fs.Read(buffer, 0, buffer.Length); string content
= Encoding.UTF8.GetString(buffer); Console.WriteLine($"FileStream
read: {content}");
}

// MemoryStream - write data in memory using (MemoryStream ms =
new MemoryStream()) { byte[] data =
Encoding.UTF8.GetBytes("MemoryStream data"); ms.Write(data, 0,
data.Length); Console.WriteLine($"Bytes written to MemoryStream:
{ms.Length}"); ms.Position = 0; string result =
Encoding.UTF8.GetString(ms.ToArray());
Console.WriteLine($"MemoryStream content: {result}");
}
}
}

```

[Console Output – Exercise 25]

```

FileStream read: Hello from FileStream!
Bytes written to MemoryStream: 17
MemoryStream content: MemoryStream data

```

Exercise 26: Race Conditions with Multi-threading

Program.cs

```

using System; using System.Threading; class Program { static
int counter = 0; static readonly object _lock = new object();
static void Increment() { for (int i = 0; i < 10000; i++) {
lock (_lock) // prevents race condition { counter++; }
}
}
}

```

```
static void Main()
{
    Thread t1 = new Thread(Increment);
    Thread t2 = new Thread(Increment);
    Thread t3 = new Thread(Increment);
    t1.Start(); t2.Start(); t3.Start();
    t1.Join(); t2.Join(); t3.Join();

    // Without lock: result would be < 30000 (race condition)
    // With lock: always exactly 30000
    Console.WriteLine($"Counter (with lock): {counter}");
}
}
```

[Console Output – Exercise 26]

```
Counter (with lock): 30000
```

```
Without lock (race condition), typical output:
```

```
Counter (no lock): 27843 <- incorrect, varies each run
```

Exercise 27: Simulate and Resolve a Deadlock

Program.cs

```

using System; using
System.Threading; class Program {
static readonly object lockA =
new(); static readonly object lockB
= new();
// Deadlock-safe version using Monitor.TryEnter static void Thread1() { bool
gotA = false, gotB = false; try { Monitor.TryEnter(lockA, 500, ref gotA); if
(!gotA) { Console.WriteLine("T1: could not get lockA - skipping"); return; }
Console.WriteLine("T1: acquired lockA");
Thread.Sleep(100); Monitor.TryEnter(lockB, 500, ref gotB); if (!gotB) {
Console.WriteLine("T1: could not get lockB - skipping"); return; }
Console.WriteLine("T1: acquired lockB - doing work"); } finally { if (gotB)
Monitor.Exit(lockB); if (gotA) Monitor.Exit(lockA); } } static void Thread2()
{ bool gotB = false, gotA = false; try { Monitor.TryEnter(lockB, 500, ref
gotB); if (!gotB) { Console.WriteLine("T2: could not get lockB - skipping");
return; } Console.WriteLine("T2: acquired lockB");
Thread.Sleep(100); Monitor.TryEnter(lockA, 500, ref gotA); if (!gotA) {
Console.WriteLine("T2: could not get lockA - skipping"); return; }
Console.WriteLine("T2: acquired lockA - doing work"); } finally { if (gotA)
Monitor.Exit(lockA); if (gotB) Monitor.Exit(lockB); } } static void Main() {
Thread t1 = new(Thread1); Thread t2 = new(Thread2);
t1.Start(); t2.Start(); t1.Join(); t2.Join();
Console.WriteLine("Main: both threads finished (no
deadlock)");
}
}

```

```
T1: acquired lockA
T2: acquired lockB
T1: could not get lockB - skipping
T2: acquired lockA - doing work
Main: both threads finished (no deadlock)
Note: output order may vary per run. TryEnter times out after
500ms instead of waiting forever, so deadlock is avoided.
```


Exercise 28: Log with System.Diagnostics.Trace

Program.cs

```
using System; using System.Diagnostics; using System.IO; class
Logger { private readonly TextWriterTraceListener _fileListener;
public Logger(string logFile) { _fileListener = new
TextWriterTraceListener(logFile);
Trace.Listeners.Add(_fileListener); Trace.AutoFlush = true; }
public void Log(string level, string message) { string entry =
$" [{DateTime.Now:yyyy-MM-dd HH:mm:ss}] [{level}] {message}";
Console.WriteLine(entry); // also to console
Trace.WriteLine(entry); // also to file } public void Close() =>
_fileListener.Close(); } class Program { static void Main() {
Logger logger = new Logger("app.log"); logger.Log("INFO",
"Application started"); logger.Log("WARN", "Low disk space
detected"); logger.Log("ERROR", "Failed to connect to
database"); logger.Log("INFO", "Application shutting down");
logger.Close(); Console.WriteLine("Log written to app.log");
}
}
```

[Console Output + app.log – Exercise 28]

```
[2024-08-01 10:23:01] [INFO] Application started
[2024-08-01 10:23:01] [WARN] Low disk space detected
[2024-08-01 10:23:01] [ERROR] Failed to connect to database [2024-
08-01 10:23:01] [INFO] Application shutting down Log written to
app.log app.log file contains the same 4 lines.
```

Exercise 29: Sanitize Input and Prevent XSS

Program.cs

```
using System; using System.Net; class Program { static
string SanitizeInput(string input) { // HtmlEncode
converts < > & " ' to HTML entities return
WebUtility.HtmlEncode(input);
}
static void Main() {
string[] testInputs
= {
"Hello, World!",
"<script>alert('XSS')</script>",
"<b>Bold text</b>", "User & Admin
<test@mail.com>" }; foreach
(string raw in testInputs) {
string safe = SanitizeInput(raw);
Console.WriteLine($"Raw: {raw}");
Console.WriteLine($"Safe:
{safe}"); Console.WriteLine();
}
}
}
```

[Console Output – Exercise 29]

```
Raw: Hello, World!
```

Safe: Hello, World!

Raw: <script>alert('XSS')</script> Safe:
<script>alert('XSS')</script>

Raw: Bold text

Safe: Bold text

Raw: User & Admin <test@mail.com>

Safe: User & Admin <test@mail.com>

Exercise 30: CRUD Operations using ADO.NET

Requires SQL Server running locally with a database named CompanyDB and an Employees table. Run the setup script below first.

SQL – Create table (run in SSMS first)

```
CREATE DATABASE CompanyDB;
GO USE
CompanyDB;
GO
CREATE TABLE Employees (
Id INT IDENTITY PRIMARY KEY,
Name NVARCHAR(100) NOT NULL,
Position NVARCHAR(100),
Salary DECIMAL(10,2)
);
```

Program.cs – full CRUD with ADO.NET

[illegible]

